实验九. 基于 yolov5 的红绿灯识别

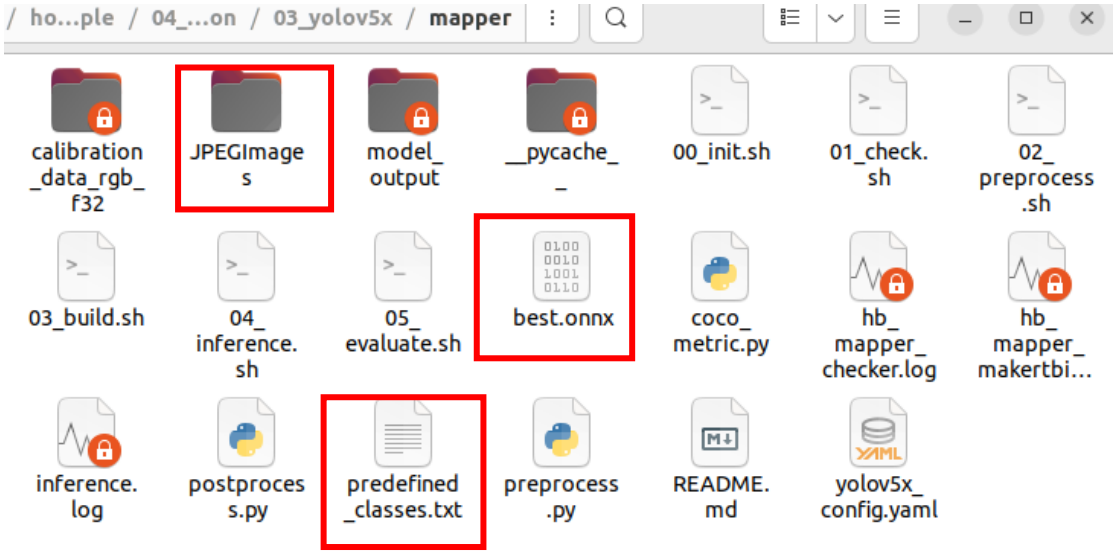
第三部分 模型转换及运行

4. 模型量化

打开虚拟机

① 将标注数据集文件夹 JPEGImages 和生成的 best.onnx 模型、VOC2007 文件夹中的 predefined_classes.txt 拷贝到以下文件夹中

/home/horizon_x5_open_explorer_v1.2.8-py310_20240926/samples/ai_toolchain/horizon_model_convert_sample/04_detection/03_yolov5x/mapper



修改 01_check.sh

```
10 set -e -v
11 cd $(dirname $0) || exit
12
13 model_type="onnx"
14 onnx_model="./best.onnx"
15 march="bayes-e"
16
17 hb_mapper checker --model-type ${model_type} \
18                    --model ${onnx_model} \
19                    --march ${march}
```

修改 02_preprocess.sh

```
set -e -v
cd $(dirname $0) || exit

python3 ../../../data_preprocess.py \
  --src_dir ./JPEGImages \
  --dst_dir ./calibration_data_rgb_f32 \
  --pic_ext .rgb \
  --read_mode opencv \
  --saved_data_type float32
```

修改 03_build.sh

```
set -e -v
cd $(dirname $0)
config_file="./yolov5x_config.yaml"
model_type="onnx"
# build model
hb_mapper makertbin --config ${config_file}  \
                    --model-type  ${model_type}
```

把 yolov5x_config.yaml 上传替换。

② 在虚拟机中，打开一个 terminal 窗口，运行：

cd horizon_x5_open_explorer_v1.2.8-py310_20240926/

sudo bash run_docker.sh /data cpu

```
originbot@originbot-virtual-machine:~$ cd horizon_x5_open_explorer_v1.2.8-py310_
20240926/
originbot@originbot-virtual-machine:~/horizon_x5_open_explorer_v1.2.8-py310_2024
0926$ sudo bash run_docker.sh /data cpu
[sudo] password for originbot:
root@c2b50851f8e5:/open_explorer#
```

cd samples/ai_toolchain/horizon_model_convert_sample/04_detection/03_yolov5x/mapper

sh 01_check.sh

```
2025-04-03 15:00:47,322 INFO The specified model compilation architecture: bernoulli2.
2025-04-03 15:00:47,322 INFO The specified model compilation optimization parameters:
2025-04-03 15:00:47,324 INFO Start to prepare the onnx model.
2025-04-03 15:00:47,335 INFO Input ONNX Model Information:
ONNX IR version:          6
Opset version:            ['ai.onnx v11', 'horizon v1']
Producer:                 pytorch v1.13.1
Domain:                   None
Version:                  None
Graph input:
    images:               shape=[1, 3, 672, 672], dtype=FLOAT32
Graph output:
    output:               shape=[1, 84, 84, 21], dtype=FLOAT32
    370:                  shape=[1, 42, 42, 21], dtype=FLOAT32
    372:                  shape=[1, 21, 21, 21], dtype=FLOAT32
2025-04-03 15:00:47,617 INFO End to prepare the onnx model.
2025-04-03 15:00:47,641 INFO Saving model to: ./.hb_check/original_float_model.onnx.
2025-04-03 15:00:47,641 INFO Start to optimize the onnx model.
2025-04-03 15:00:48,484 INFO End to optimize the onnx model.
```

```
/model.23/cv3/Conv                                BPU  id(0)     HzSQuantizedConv

/model.23/cv2/Conv                                BPU  id(0)     HzSQuantizedConv

/model.23/Concat                                  BPU  id(0)     Concat

/model.23/act/LeakyRelu                           BPU  id(0)     HzLeakyRelu

/model.23/cv4/conv/Conv                           BPU  id(0)     HzSQuantizedConv

/model.23/cv4/act/LeakyRelu                       BPU  id(0)     HzLeakyRelu

/model.24/m.0/Conv                                BPU  id(0)     HzSQuantizedConv

/model.24/m.1/Conv                                BPU  id(0)     HzSQuantizedConv

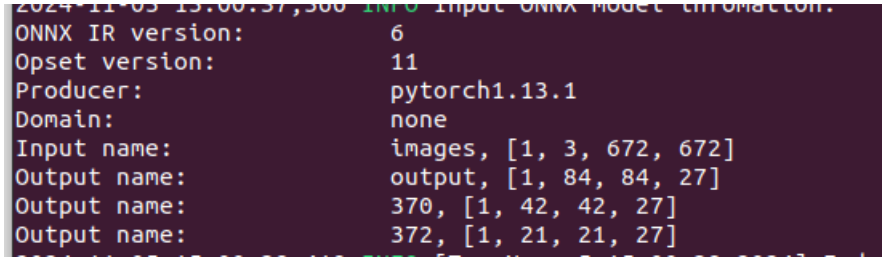
/model.24/m.2/Conv                                BPU  id(0)     HzSQuantizedConv
2024-11-10 13:20:55,374 INFO [Sun Nov 10 13:20:55 2024] End to Horizon NN Model Convert.
2024-11-10 13:20:55,378 INFO ONNX model output num : 3
2024-11-10 13:20:55,391 INFO End model checking....
root@525638007da7:/open_explorer/ddk/samples/ai_toolchain/horizon_model_convert_sample/04_detection/03_yolov5s/mapper#
```

sh 02_preprocess.sh

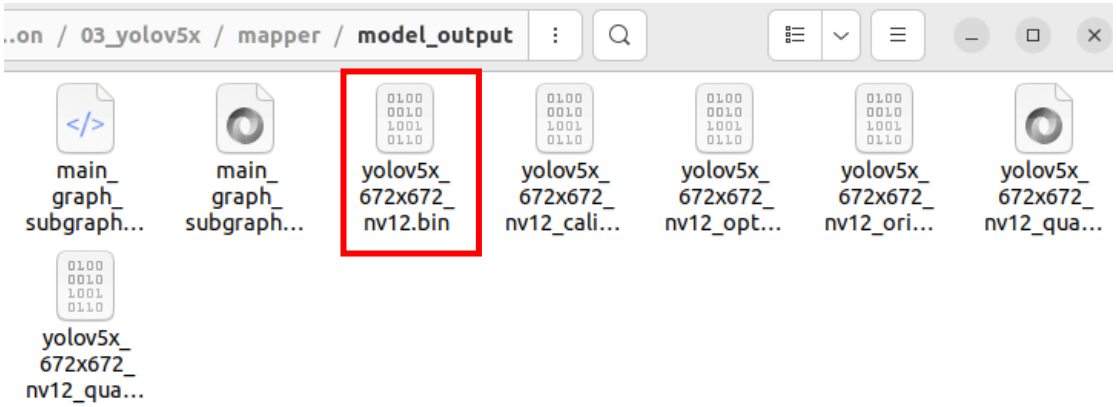
```
le/04_detection/03_yolov5x/mapper# sh 02_preprocess.sh
cd $(dirname $0) || exit

python3 ../../../data_preprocess.py \
  --src_dir ./JPEGImages \
  --dst_dir ./calibration_data_rgb_f32 \
  --pic_ext .rgb \
  --read_mode opencv \
  --saved_data_type float32
Regular preprocess
Init 2 processes
write:./calibration_data_rgb_f32/2024-11-04-19_24_40_698775.rgb
write:./calibration_data_rgb_f32/2024-11-04-19_27_03_740532.rgb
write:./calibration_data_rgb_f32/2024-11-04-19_27_04_828951.rgb
```

sh 03_build.sh



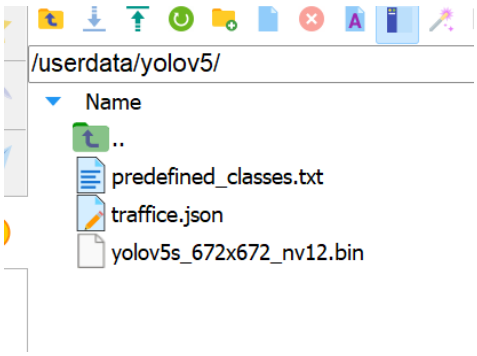
此时会在 model_output 中产生一个 bin 文件，将该文件复制，黏贴到本地计算机。



5. 上板模型部署及运行

1）准备工作

远程连接小车，在/userdata 目录下，新建 yolov5 文件夹，上传 yolov5x_672x672_nv12.bin、predefined_classes.txt 和 traffice.json 三个文件到该目录下



附 traffice.json

```
{
        "model_file": "/userdata/yolov5/yolov5x_672x672_nv12.bin",
        "model_name": "yolov5x_672x672_nv12",
        "task_num": 2,
        "dnn_Parser": "yolov5x",
        "model_output_count": 3,
        "class_num": 3,
        "cls_names_list": "/userdata/yolov5/predefined_classes.txt",
```

```
        "strides": [8, 16, 32],
        "anchors_table": [[[10, 13], [16, 30], [33, 23]], [[30, 61], [62, 45],
[59, 119]], [[116, 90], [156, 198], [373, 326]]],
        "score_threshold": 0.3,
        "nms_threshold": 0.5,
        "nms_top_k": 5000,
        "output_order": [0,1,2]
}
```

2）实际验证

在第一个窗口，输入

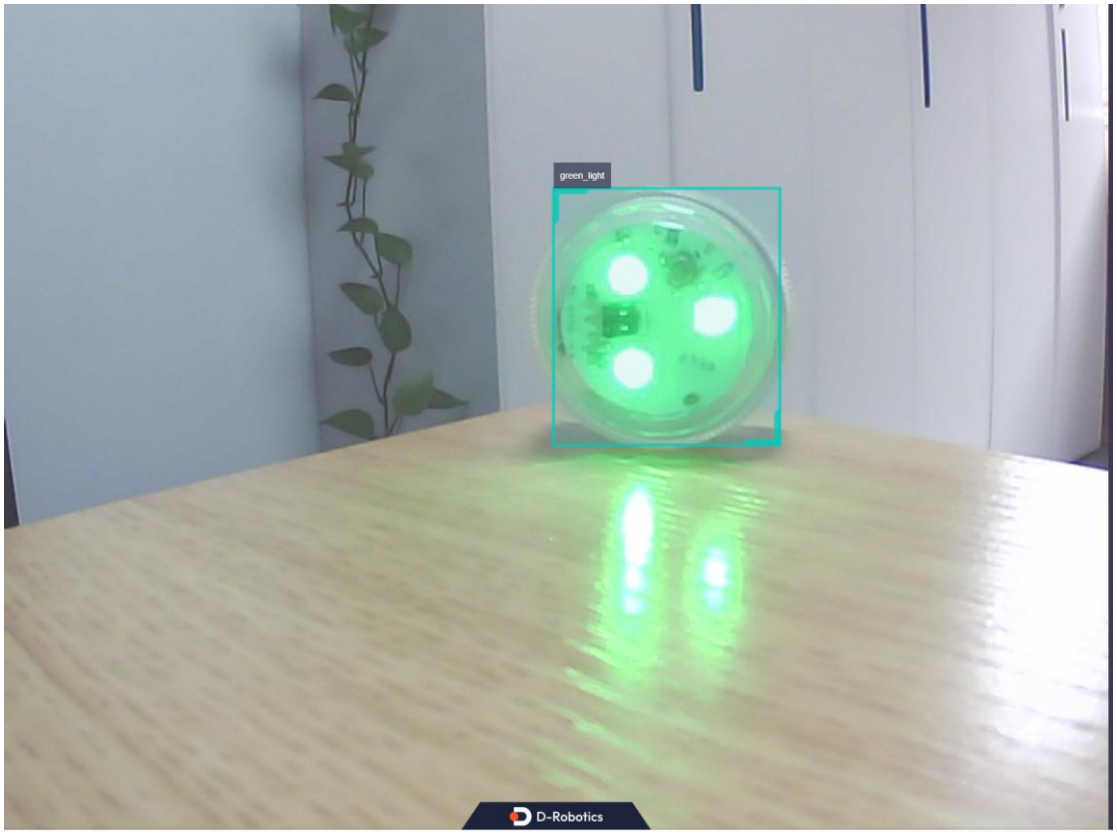
export CAM_TYPE=usb

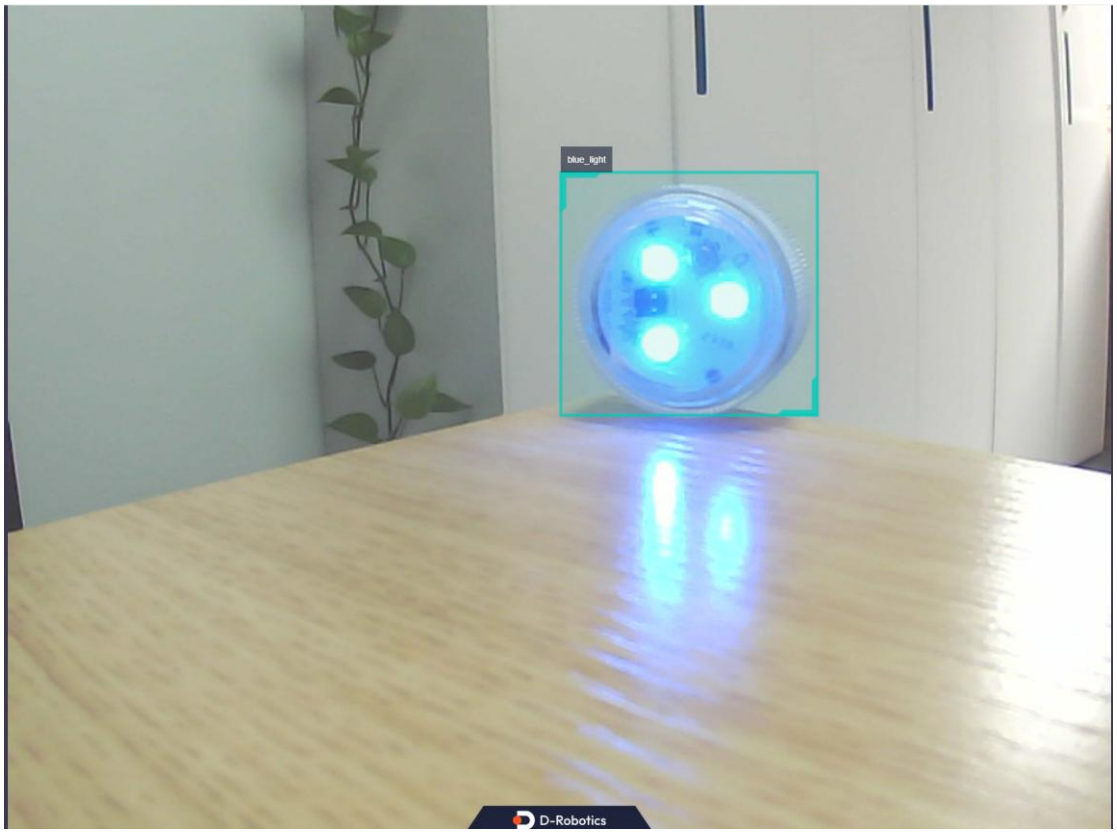
ros2 launch dnn_node_example dnn_node_example.launch.py

dnn_example_config_file:=/userdata/yolov5/traffice.json

打开红绿灯，放置在小车前方，通过网页观察是否推理正确







实际使用

1）在 /userdata/dev_ws/install/lanedetection/share/lanedetection/目录下，新建 lanunch 文件夹，上传 dnn_node_yolo.launch.py 到这个目录下

2）启动底盘、摄像头和激光雷达

ros2 launch originbot_bringup originbot.launch.py use_camera:=true use_lidar:=true

3）第 2 个窗口，启动 yolo 识别红绿灯的程序

ros2 launch lanedetection dnn_node_yolo.launch.py

4）第 3 个窗口，查看话题（topic）和数据

ros2 topic list

```
root@ubuntu:~# ros2 topic list
/camera_info
/cmd_vel
/hbmem_img0x2107020403080120021609080306270d
/odom
/originbot_status
/parameter_events
/rosout
/scan
/tf
/tf_static
/traffic_sign
/ydlidar_ros2_driver_node/transition_event
root@ubuntu:~#
```

查看数据

ros2 topic echo /traffice_sign

```
targets:
- type: "red_light\r"
  track_id: 0
  rois:
  - type: "red_light\r"
    rect:
      x_offset: 612
      y_offset: 55
      height: 41
      width: 41
      do_rectify: false
    confidence: 0.41854217648506165
  attributes: []
  points: []
  captures: []
disappeared_targets: []
---
```

红灯 red_light

```
targets:
- type: green_light
  track_id: 0
  rois:
  - type: green_light
    rect:
      x_offset: 652
      y_offset: 67
      height: 32
      width: 31
      do_rectify: false
    confidence: 0.7247857451438904
  attributes: []
  points: []
  captures: []
disappeared_targets: []
```

绿灯 green_light

查看数据类型

ros2 topic type /traffic_sign

```
^Croot@ubuntu:~# ros2 topic type /traffic_sign
ai_msgs/msg/PerceptionTargets
root@ubuntu:~#
```

根据 view_traffic.py 修改 lanedetection.py

附录 view_traffic.py

```
1.  #!/usr/bin/env python
2.  # -*- coding: UTF-8 -*-
3.
4.  import rclpy
5.  from rclpy.node import Node
6.  from std_msgs.msg import Bool
7.  from ai_msgs.msg import PerceptionTargets
8.
9.  class ViewTraffic(Node):
10.     def __init__(self,name):
11.         super().__init__(name)
12.         self.get_logger().info("Traffic_sign is OK")
13.
14.         self.traffic_sub= self.create_subscription(PerceptionTargets,
15.             "/traffice_sign", self.traffic_callback, 10)
16.
17.     def traffic_callback(self, msg):
18.          for boxes in msg.targets:
19.             if boxes.rois[0].confidence > 0.3 and boxes.rois[0].type == 'gre
    en_light\r':
20.                 print('green_light')
21.             elif boxes.rois[0].confidence > 0.3 and boxes.rois[0].type == 'r
    ed_light':
```

```
                print('red_light')

def main(args=None):
    rclpy.init(args=args)
    trafficview = ViewTraffic("View_Traffic")
    rclpy.spin(trafficview)
    trafficview.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

附录 dnn_node_yolo. launch. py

```
import os
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    dnn_node = Node(
        package='dnn_node_example',
        executable='example',
        name='dnn_node',
        output='screen',
        arguments=[
            '--ros-args', '--log-level', 'warn',
            '--ros-args', '-p', 'feed_type:=1',
            '--ros-args', '-p', 'dump_render_img:=0',
            '--ros-args', '-p', 'is_shared_mem_sub:=1',
            '--ros-args', '-
p', 'config_file:=/userdata/yolo11/yolov11workconfig.json',
            '--ros-args', '-p', 'msg_pub_topic_name:=traffice_sign'
        ]
    )
    return LaunchDescription([
        dnn_node,
    ])
```